

CulinaTwin: A Multimodal AI Architecture for Inventory-Aware Cooking Assistance, Flavor Substitution, and Real-Time Culinary Execution

VU Anh Duy

Department of Computer Science
Université Claude Bernard Lyon 1, France
anh-duy.vu@etu.univ-lyon1.fr

Technical Report · May 3, 2026

Acronym: CulinaTwin

Abstract

This report presents the technical and research blueprint for *CulinaTwin*, an AI system for kitchen inventory management and intelligent cooking assistance. The architecture fuses document AI, computer vision for refrigerator understanding, graph-based flavor reasoning, traditional culinary philosophy, and edge-deployed real-time execution monitoring into a unified pipeline. The core objective is to transform heterogeneous sensory inputs—receipt images, refrigerator photographs, and live cooking video—into structured inventory knowledge, actionable recipe suggestions, chemically grounded substitutions, seasonally and energetically balanced meal plans, and real-time execution feedback. This document serves two complementary purposes: (i) to define a rigorous research and implementation roadmap, and (ii) to serve as a collaboration invitation for computer vision researchers, backend engineers, data scientists, culinary domain experts, and edge-AI practitioners.

Keywords: multimodal AI, food computing, inventory management, document understanding, flavor graphs, edge inference, culinary philosophy.

Contents

1	Introduction	4
2	Related Work	4
2.1	Document AI for Receipt Understanding	4
2.2	Vision for Refrigerator and Pantry Understanding	4
2.3	Flavor Graphs and Ingredient Substitution	5
2.4	Culinary Philosophy and Seasonality	5
2.5	Real-Time Cooking Assistance at the Edge	5
3	System Architecture	5
3.1	Design Goals	5
3.2	Logical Architecture	5
3.3	Physical Deployment Architecture	5
4	Module 1: Vision and Ingestion Pipeline	6

4.1	Design	6
4.2	Illustrative Example	6
5	Module 2: Dynamic Inventory and Flavor Matrix	7
5.1	Design	7
5.2	Illustrative Example	7
6	Module 3: Culinary Philosophy Integration	7
6.1	Design	7
6.2	Illustrative Example	8
7	Module 4: Real-Time Live Cooking Assistant	8
7.1	Design	8
7.2	Illustrative Example	9
8	Data Model	9
8.1	PostgreSQL Core Tables	9
8.2	Neo4j Core Relationships	10
9	API Specification	10
10	Evaluation Metrics	11
11	Implementation Roadmap	11
11.1	Phase 1 — MVP Foundation	11
11.2	Phase 2 — Fridge Vision	11
11.3	Phase 3 — Flavor Graph	11
11.4	Phase 4 — Culinary Philosophy	11
11.5	Phase 5 — Real-Time Assistant	12
11.6	Phase 6 — Production Hardening	12
12	Project Timeline	12
13	Collaboration Opportunities	12
14	Conclusion	12

List of Figures

1	High-level logical architecture of <i>CulinaTwin</i>	5
2	Gantt chart for the six-phase implementation plan.	12

List of Tables

1	Required API endpoints for <i>CulinaTwin</i>	11
2	Evaluation metrics by system component.	11

1 Introduction

Modern cooking workflows are fragmented. Grocery purchases remain trapped in paper receipts; fridge contents are largely invisible to digital systems; substitution decisions are often manual and imprecise; and recipe generation rarely accounts for seasonality, ingredient chemistry, or energetic balance. Existing food-tech solutions typically address only one narrow problem at a time: OCR for receipts, object detection for storage, recipe recommendation, or timers for cooking. *CulinaTwin* proposes a unified system that treats the kitchen as a continuously updated *digital twin*.

The core insight is that every purchase, scan, recipe execution, and cooking event constitutes a structured signal that can update a persistent culinary memory. This memory then drives four downstream capabilities:

1. automatic inventory tracking and depletion modelling,
2. substitution-aware recipe generation,
3. culturally grounded menu balancing, and
4. real-time expeditor assistance during cooking execution.

The initial design is informed by three independent architectural drafts and a research synthesis. A common consensus emerges across these materials: receipt OCR should be handled by a layout-aware document model such as LayoutLMv3 [Huang et al., 2022]; fridge scanning should combine detection, segmentation, and monocular depth estimation; inventory should be stored in PostgreSQL and linked to a graph database for flavor relationships; substitution should be grounded in chemical similarity and role preservation; culinary reasoning should encode Yin–Yang and Five Elements constraints; and live cooking should be executed at the edge using low-latency CNNs optimized with TensorRT or ONNX Runtime [Corporation, 2023, Microsoft, 2023].

2 Related Work

2.1 Document AI for Receipt Understanding

Receipt digitization systems increasingly combine OCR with layout-aware transformers. Plain OCR is insufficient because the semantic meaning of a receipt depends on spatial relations: item names, quantities, unit prices, totals, and taxes are arranged in columnar structures. LayoutLMv3 [Huang et al., 2022] is a strong baseline because it jointly encodes text and image patches, demonstrating state-of-the-art performance on receipt understanding benchmarks.

2.2 Vision for Refrigerator and Pantry Understanding

Inventory-aware fridge perception remains harder than standard object detection because the system must reason about partial occlusion, irregular shapes, mixed containers, and scale ambiguity. The most practical approach is a two-stage pipeline: instance detection or segmentation followed by volume and mass estimation. YOLOv8 [Ultralytics, 2023] is attractive for fast edge deployment, while Mask R-CNN [He et al., 2017] remains a strong option for precise masks in offline or semi-offline settings. Monocular depth estimation and shelf-based calibration can produce usable mass estimates when combined with density priors [Ranftl et al., 2021, Ege and Yanai, 2017].

2.3 Flavor Graphs and Ingredient Substitution

Ingredient substitution cannot be solved reliably by simple synonym lookup. The state of the art increasingly points toward graph-based formulations in which ingredients are linked to flavor molecules, recipe co-occurrence, and substitute relationships. FlavorDB [Garg et al., 2018] provides a chemically grounded substrate, while ingredient-compound graph embeddings [Park et al., 2021] and recipe similarity networks [Teng et al., 2012] support more nuanced matching than rule-based tables.

2.4 Culinary Philosophy and Seasonality

For East Asian culinary systems, food is not only a biochemical input but also an energetic one. Yin–Yang and Five Elements models can be encoded as structured metadata and used as constraints in recommendation generation [Liu et al., 2015, Zhang and Wang, 2019]. In practice, these principles become effective when translated into machine-readable fields such as thermal property, taste category, element, seasonality, and contraindications.

2.5 Real-Time Cooking Assistance at the Edge

A cooking assistant that only plans is insufficient; it must monitor and intervene during live cooking. This requires edge inference, low latency, and robust state tracking. Lightweight CNNs optimized through ONNX Runtime or TensorRT can detect browning, caramelization, emulsion breakdown, and other state transitions before humans perceive failure [Sun et al., 2020, Chen et al., 2021].

3 System Architecture

3.1 Design Goals

CulinaTwin is designed to satisfy the following goals:

1. Convert unstructured purchase and camera data into structured inventory knowledge.
2. Maintain an accurate and temporally updated stock model.
3. Suggest recipe substitutions that preserve both chemistry and cooking function.
4. Balance menus with seasonal, nutritional, and philosophical constraints.
5. Monitor live cooking and prevent critical failures before they occur.

3.2 Logical Architecture

Figure 1 depicts the high-level logical architecture. The system is organized into four sequential layers: ingestion, knowledge, reasoning, and execution.

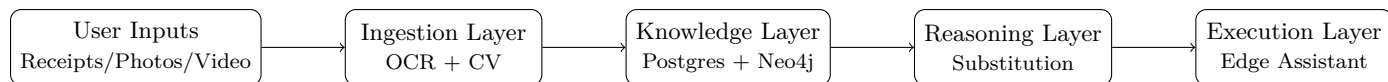


Figure 1: High-level logical architecture of *CulinaTwin*.

3.3 Physical Deployment Architecture

The physical deployment separates latency-sensitive and stateful components:

- **Mobile client:** image capture, manual correction, recipe interaction.

- **API gateway:** authentication, upload routing, and rate limiting.
- **Vision services:** OCR worker, fridge vision worker, edge cooking worker.
- **Data layer:** PostgreSQL for transactions, Neo4j for flavor knowledge, object storage for raw images.
- **Event layer:** Kafka for inventory events and live cooking alerts.
- **Model layer:** model registry, evaluation pipelines, and retraining jobs.

4 Module 1: Vision and Ingestion Pipeline

4.1 Design

The ingestion subsystem consists of two parallel perception branches.

Branch A: Receipt Understanding. The recommended pipeline is:

Preprocessing → OCR → Layout Parsing → Schema Extraction → Canonicalization → DB Insert

Preprocessing steps include deskewing, denoising, contrast normalization, and perspective correction. OCR can be handled by TrOCR, PaddleOCR, or Tesseract as a fallback. LayoutLMv3 [Huang et al., 2022] then receives OCR tokens and bounding boxes to recover receipt structure. The output must be deterministic JSON with fields **ingredient**, **quantity**, **unit**, **price**, and **confidence** [Grover et al., 2022].

Branch B: Fridge Scanning. The recommended pipeline is:

Capture → Detection/Segmentation → Calibration → Depth Inference → Volume → Mass Estimation

YOLOv8-seg [Ultralytics, 2023] is the default edge model; Mask R-CNN [He et al., 2017] may be used when precision takes priority over latency. Monocular depth estimation provides relative 3D cues; shelf geometry or a calibration object resolves scale; density priors convert volume into estimated mass [Ranftl et al., 2021, Ege and Yanai, 2017].

Key design principle: all input modalities converge on the same inventory schema, avoiding fragmented data silos and making downstream reasoning deterministic.

4.2 Illustrative Example

A user photographs a receipt after shopping. The OCR branch extracts:

Tomates cerises bio	250 g	3.45 EUR
Lait demi-écrémé	1 L	1.29 EUR
Chou vert	1.2 kg	2.10 EUR

LayoutLMv3 identifies line items; the schema extractor normalizes them into:

```
[
  {"ingredient": "tomatoes_cherry", "quantity": 250, "unit": "g", "price": 3.45},
  {"ingredient": "semi_skimmed_milk", "quantity": 1000, "unit": "ml", "price": 1.29},
  {"ingredient": "green_cabbage", "quantity": 1200, "unit": "g", "price": 2.10}
]
```

Two days later, a fridge scan detects the cabbage with a partial mask. After calibration and mass estimation, the cabbage entry is updated from 1,200 g to 820 g. The UI reflects: *“Green cabbage: 820 g remaining · last seen 18:41 · confidence 0.87.”*

5 Module 2: Dynamic Inventory and Flavor Matrix

5.1 Design

Inventory management is split into two complementary layers.

Relational layer. PostgreSQL stores transactional state: items, quantities, units, purchase dates, expiration dates, scan timestamps, and provenance. This layer is optimized for correctness, ACID semantics, and auditability.

Graph layer. Neo4j stores the flavor matrix: ingredients, compounds, tastes, roles, pairing strength, and substitute relations. FlavorDB-style molecular data [Garg et al., 2018] is a natural seed source. Graph embeddings derived from ingredient-compound relations [Park et al., 2021] are more expressive than static pair tables and align well with recipe similarity research [Teng et al., 2012].

The substitution engine must preserve the *functional role* of the missing ingredient. For example, if the ingredient is an acidulant, the substitute must still lower pH; if it is an emulsifier, it must still stabilize the mixture. A practical scoring function is:

$$s = \alpha \cdot \text{chem_sim} + \beta \cdot \text{role_match} + \gamma \cdot \text{texture_sim} + \delta \cdot \text{recipe_ctx} + \varepsilon \cdot \text{availability} \quad (1)$$

where $\alpha, \beta, \gamma, \delta, \varepsilon \geq 0$ are task-specific weights.

The auto-deduction pipeline is event-driven. When a recipe is executed, the system emits a `recipe.completed` event. Consumers update stock, flag depleted items, and promote near-expiry items for priority use—enabling waste reduction and reactive stock management.

5.2 Illustrative Example

The user wants to prepare a Vietnamese noodle soup requiring ginger, which is absent from the fridge. The substitution engine queries Neo4j and scores galangal, turmeric, and lemongrass as candidates. Galangal wins because it shares more aromatic compounds and preserves the pungent warming role (see eq. (1)). The UI displays:

Missing: ginger. Recommended substitute: galangal (95.8% compatible). Rationale: shared volatile compounds, equivalent warming profile, similar fibrous structure.

After cooking, the system receives a completion event and decrements inventory entries. Fermented tofu expiring in two days is promoted to the top of the recommendation queue, and the user sees: “Use fermented tofu tonight; 200 g remaining; expires soon.”

6 Module 3: Culinary Philosophy Integration

6.1 Design

CulinaTwin treats culinary philosophy as a structured constraint system rather than decorative metadata. Each ingredient carries the following fields:

- **Thermal property:** Yin, Yang, or Neutral [Liu et al., 2015];
- **Five Elements label:** Metal, Wood, Water, Fire, or Earth;
- **Primary and secondary taste profiles** [Zhang and Wang, 2019];
- **Seasonality profile** (by geographic region);

- **Organ affinities and contraindications** where relevant.

The recommendation engine consumes three context sources: inventory state, geolocation, and real-time weather. Hot and humid ambient conditions push the objective toward cooling and hydrating ingredients; cold conditions shift toward warming ingredients.

For menu generation, an LLM is used as a *language executor*, not as the source of truth. The constrained planning pipeline is as follows:

1. Generate candidate dishes from inventory and seasonality constraints.
2. Score candidates with energetic and nutritional objectives.
3. Solve the final selection with CP-SAT or heuristic search.
4. Invoke the LLM to verbalize and format the selected menu.

This architecture is safer and more reproducible than unconstrained prompt-only generation.

6.2 Illustrative Example

It is winter in Lyon; ambient conditions are cold and dry. The user requests a plant-based dinner. The planner raises the target toward warming, grounding, and mildly moistening dishes, and rejects a raw salad branch as too cooling for the ambient context. The proposed menu is:

- braised cabbage with ginger and white pepper,
- fermented black bean side,
- warm grain bowl with legumes, and
- mild fruit dessert to avoid over-heating the menu.

The UI explanation is structured rather than mystical:

The menu balances Yang main energy with a Yin counterweight, includes Metal and Water elements, remains within the seasonal target for winter, and uses only ingredients currently in stock.

7 Module 4: Real-Time Live Cooking Assistant

7.1 Design

The live cooking assistant runs at the edge. Cloud inference is too slow for thermal failure windows measured in seconds. The recommended stack is as follows:

- **Edge runtime:** NVIDIA Jetson or equivalent NPU-capable device;
- **Inference layer:** ONNX Runtime [Microsoft, 2023] or TensorRT [Corporation, 2023];
- **Model family:** lightweight CNNs (MobileNetV3 or compact ResNet variants);
- **Task layer:** DAG-based scheduler for multi-station coordination;
- **Alert layer:** micro-notifications for heat reduction, stirring, flipping, or plating.

The detection heads track five cooking state indicators:

1. Maillard reaction trajectory and browning gradient [Sun et al., 2020];
2. caramelization stage of sugars;
3. emulsion stability and phase separation;
4. smoke-risk indicators for oil; and
5. doneness state for proteins and plant-based analogues [Chen et al., 2021].

Frame capture, preprocessing, inference, and alert dispatch must remain below 100 ms per cycle. TensorRT is recommended for deployment because it enables graph fusion, precision calibration, and high-throughput serialized engines on Jetson-class hardware [Corporation, 2023].

The scheduler parses each recipe into a directed acyclic graph (DAG):

$$\text{Task}_i \rightarrow \text{Task}_j \iff \text{Task}_j \text{ depends on } \text{Task}_i \quad (2)$$

Each edge encodes a precedence constraint. The optimizer computes the critical path and continuously recomputes the remaining timeline as live state updates arrive.

7.2 Illustrative Example

The user prepares a three-component dish: seared king oyster mushrooms, plant-based hollandaise, and crispy potatoes. The assistant schedules mushrooms first, sauce second, and potatoes in parallel.

During execution, the mushroom surface hits the browning threshold early. The edge model flags:

Browning reached 86%. Reduce heat now.

Ten seconds later, the sauce model detects early emulsion instability and issues:

Whisk immediately. Free oil droplets detected at the rim.

The scheduler shifts remaining tasks by two minutes, reorders the plating sequence, and keeps the full dish synchronized, providing professional-grade timing guidance.

8 Data Model

8.1 PostgreSQL Core Tables

```

ingredients(
  ingredient_id      UUID PRIMARY KEY,
  canonical_name     TEXT,
  category           TEXT,
  density_g_per_ml   NUMERIC,
  thermal_property    TEXT,          -- Yin | Yang | Neutral
  five_element       TEXT,          -- Metal | Wood | Water | Fire | Earth
  primary_taste      TEXT,
  seasonality        JSONB
)

inventory_items(
  inventory_item_id  UUID PRIMARY KEY,
  user_id            UUID,

```

```

    ingredient_id    UUID REFERENCES ingredients,
    quantity         NUMERIC,
    unit             TEXT,
    expiry_date      DATE,
    storage_location TEXT,
    source           TEXT,          -- receipt | scan | manual
    last_seen_at     TIMESTAMP,
    confidence        NUMERIC
)

receipts(
    receipt_id       UUID PRIMARY KEY,
    vendor_name      TEXT,
    purchase_datetime TIMESTAMP,
    currency          TEXT
)

receipt_lines(
    line_id          UUID PRIMARY KEY,
    receipt_id       UUID REFERENCES receipts,
    raw_name         TEXT,
    ingredient_id    UUID REFERENCES ingredients,
    quantity         NUMERIC,
    unit             TEXT,
    unit_price       NUMERIC,
    line_total       NUMERIC,
    confidence       NUMERIC
)

recipe_executions(
    execution_id     UUID PRIMARY KEY,
    recipe_id        UUID,
    user_id          UUID,
    execution_timestamp TIMESTAMP,
    ingredients_consumed JSONB
)

```

8.2 Neo4j Core Relationships

```

(:Ingredient)-[:CONTAINS {concentration: float}]->(:Compound)
(:Ingredient)-[:PAIRS_WITH {score: float}]->(:Ingredient)
(:Ingredient)-[:SUBSTITUTES_FOR {compatibility: float}]->(:Ingredient)
(:Ingredient)-[:HAS_TEMPERATURE]->(:ThermalProperty)
(:Ingredient)-[:HAS_ELEMENT]->(:Element)
(:Ingredient)-[:HAS_TASTE]->(:Taste)

```

9 API Specification

Table 1 lists the required REST and streaming endpoints.

Table 1: Required API endpoints for *CulinaTwin*.

Endpoint	Method	Purpose
/api/receipt/parse	POST	Upload receipt image; return structured JSON.
/api/fridge/scan	POST	Upload fridge image; return detected items and estimated mass.
/api/inventory/update	POST	Manually correct quantities or import model estimates.
/api/recipe/generate	POST	Generate constrained recipes from inventory and context.
/api/substitution/query	POST	Return ranked substitutes for a missing ingredient.
/api/cooking/stream	WS/gRPC	Stream live cooking states and real-time alerts.
/api/plan/timeline	POST	Recompute DAG schedule for multi-station cooking.

10 Evaluation Metrics

A rigorous evaluation framework covers both ML-level and product-level metrics.

Table 2: Evaluation metrics by system component.

Component	Metrics
Receipt OCR	Field-level F1, character error rate (CER), JSON validity rate.
Fridge vision	mAP (detection), IoU (segmentation), mass MAE, calibration error.
Inventory	Deduction correctness, expiry alert precision, update latency.
Substitution	Top- k accuracy, chef acceptance rate, role preservation score.
Menu planning	Thermal balance deviation, seasonality compliance, nutritional completeness.
Live cooking	Alert lead time, false positive rate, missed-failure rate, end-to-end latency.

11 Implementation Roadmap

11.1 Phase 1 — MVP Foundation

Build receipt OCR, inventory schema, manual correction UI, and basic recipe generation. *Deliverable: users can upload receipts and view live stock.*

11.2 Phase 2 — Fridge Vision

Add YOLOv8-seg, calibration, mass estimation, and inventory reconciliation. *Deliverable: fridge scans update stock automatically.*

11.3 Phase 3 — Flavor Graph

Build the Neo4j flavor graph, ingredient embeddings, substitution ranking, and explanation layer. *Deliverable: missing ingredients receive ranked alternatives with rationale.*

11.4 Phase 4 — Culinary Philosophy

Encode Yin–Yang, Five Elements, weather context, and seasonality. Add constrained menu generation. *Deliverable: the system proposes balanced, inventory-aware menus.*

11.5 Phase 5 — Real-Time Assistant

Deploy edge inference, live state detection, and DAG scheduling. *Deliverable: the system warns before burning, sauce separation, or timing drift.*

11.6 Phase 6 — Production Hardening

Add monitoring, retraining pipelines, security review, audit logs, and canary deployment. *Deliverable: the platform is deployable in a real kitchen environment.*

12 Project Timeline

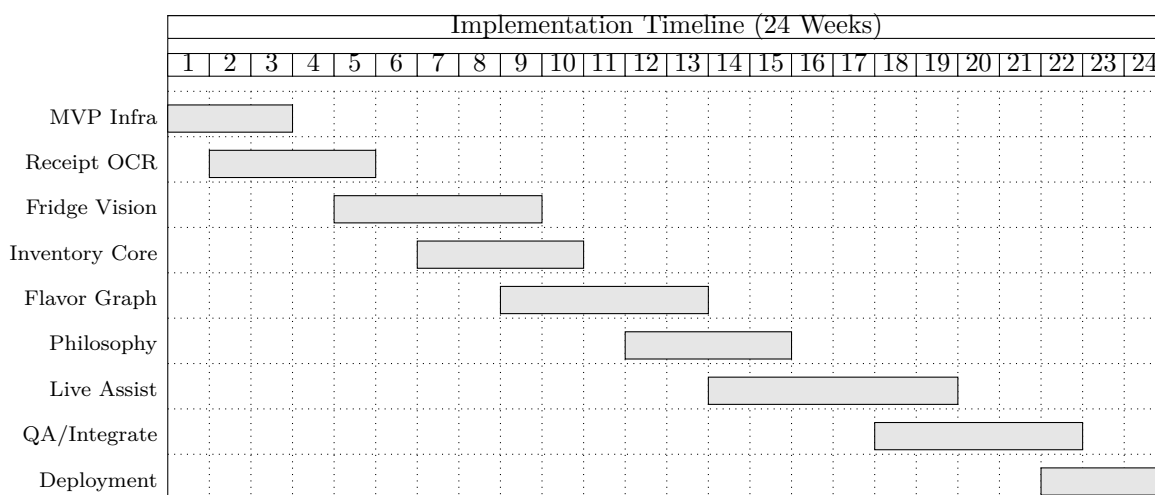


Figure 2: Gantt chart for the six-phase implementation plan.

13 Collaboration Opportunities

This project invites contributors with the following profiles:

- **Computer Vision Engineer:** OCR, detection, segmentation, depth estimation.
- **Backend Engineer:** FastAPI, PostgreSQL, Kafka, Neo4j, API design.
- **ML Engineer:** model training, fine-tuning, MLOps, evaluation pipelines.
- **Edge AI Engineer:** ONNX, TensorRT, Jetson deployment, latency optimization.
- **Culinary Domain Expert:** recipe structure, substitution logic, energetic balance.
- **Frontend / Mobile Engineer:** camera UX, inventory UI, live cooking dashboards.

14 Conclusion

CulinaTwin is not simply a recipe generator or a smart pantry application. It is a continuous culinary intelligence layer that converts the kitchen into a measurable, optimizable, and explainable system. The project’s novelty lies in the integration of document AI, visual inventory estimation, chemical flavor reasoning, philosophical constraints, and real-time execution support under a single unified architecture. This makes it a strong candidate for an academic prototype, an industry pilot, and a collaborative open-source project.

References

- L. Chen et al. Deep learning for rapid visual recognition of cooking state. *Food Research International*, 2021.
- NVIDIA Corporation. Nvidia tensorrt: Programmable inference accelerator. <https://developer.nvidia.com/tensorrt>, 2023.
- Takumi Ege and Keiji Yanai. Vision-based food weight estimation from 2d images. *arXiv preprint arXiv:1709.02219*, 2017.
- N. Garg, A. Sethupathy, R. Tuwani, et al. Flavordb: A database of flavor molecules. <https://cosylab.iiitd.edu.in/flavordb/>, 2018.
- Siddhant Grover et al. A survey on document image analysis of receipts: From ocr to information extraction. *Practical Receipt Digitization Workflow*, 2022.
- Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2961–2969, 2017.
- Yupan Huang, Tengchao Gu, Zhechi Jiang, Huaiyuan Luan, Yang Yi, Xu Wu, Lei Cui, Furu Wei, and Ming Zhou. Layoutlmv3: Pre-training for document ai with unified text and image masking. *arXiv preprint arXiv:2204.08387*, 2022.
- J. Liu et al. *Traditional Chinese Medicine Food Classification: Yin, Yang, and Neutral Properties*. Traditional Culinary Medicine References, 2015.
- Microsoft. Onnx runtime: cross-platform, high performance ml inferencing and training accelerator. <https://onnxruntime.ai/>, 2023.
- Doheum Park, Keonwoo Kim, Yong-Yeol Kim, Kwang H Jhon, and Jae-Woong Kim. Ingredient graph embeddings for food pairing and substitution. *Food Chemistry*, 351:129215, 2021.
- René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 12179–12188, 2021.
- J. Sun et al. Cooking doneness recognition and browning-stage classification using deep learning. *Journal of Food Engineering*, 2020.
- Chun-Yuen Teng, Yu-Ru Lin, and Lada A Adamic. The recipe similarity network. *Scientific Reports*, 2(1):1–7, 2012.
- Ultralytics. YOLOv8 documentation and model overview. <https://yolov8.com/>, 2023. Accessed: 2026-05-03.
- Y. Zhang and X. Wang. Five flavor mapping and the five elements in traditional asian nutrition. *Journal of Ethno-Medicine*, 2019.